

Fundamentos del plan de pruebas de software

Breve descripción:

Este Este componente aborda los conceptos, principios, niveles y tipos de pruebas de software, así como los marcos de referencia y estándares aplicables. Orienta al aprendiz en la definición del procedimiento técnico y en la construcción del plan de pruebas, incluyendo alcance, recursos, escenarios, nomenclaturas y casos de prueba.

Agosto 2026

Tabla de contenido

Introducción	4
1. Fundamentos de las pruebas de software	7
1.1. Concepto, objetivos y alcance de las pruebas de software	8
1.2. Principios, importancia e impacto del software testing	10
2. Niveles de pruebas de software y ciclo de vida.....	14
2.1. Ciclo de vida del desarrollo de software y Modelo en V	14
2.2. Pruebas unitarias, de integración, del sistema y de aceptación.....	16
3. Tipos de pruebas de software.....	20
3.1. Pruebas funcionales y no funcionales	20
3.2. Pruebas de revisión, estáticas y dinámicas	23
4. Marcos de referencia y estándares en pruebas de software	26
4.1. Estándares, técnicas y procedimientos técnicos	26
4.2. Documentación, gestión de configuración y lecciones aprendidas	28
5. Plan de pruebas de software (test plan)	32
5.1. Concepto, propósito, alcance y enfoque del plan de pruebas.....	32
5.2. Recursos, cronograma, ítems, escenarios y casos de prueba	37
Síntesis	41
Glosario	42

Referencias bibliográficas	45
Créditos	46

Introducción

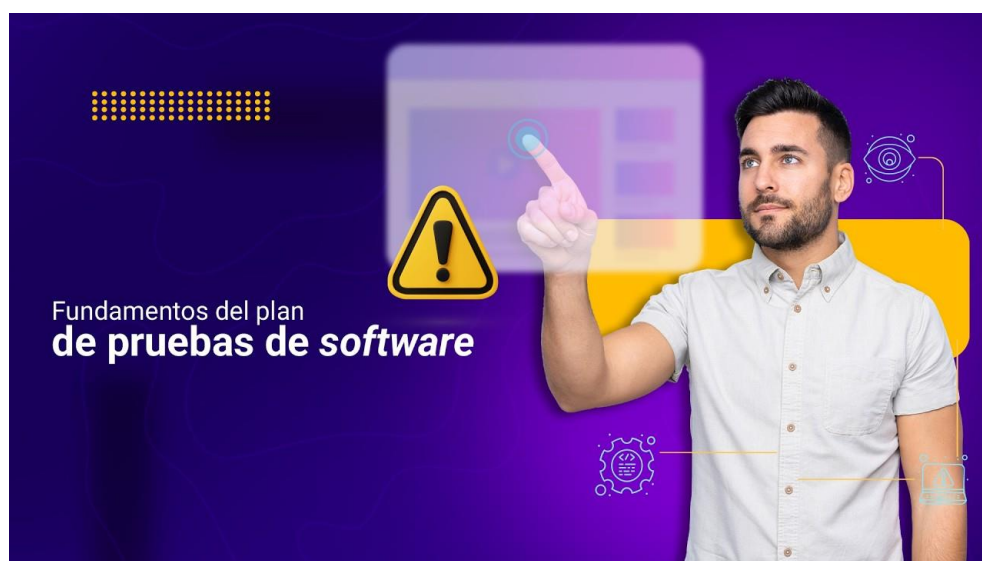
En el desarrollo de soluciones tecnológicas, la calidad del software no depende únicamente de que una aplicación funcione, sino de que responda a los requisitos definidos, sea confiable, segura y útil para las personas que la utilizarán. Por esta razón, las pruebas de software cumplen un papel fundamental dentro de los proyectos de desarrollo, mantenimiento e implementación de sistemas. Este componente formativo aborda los fundamentos necesarios para comprender qué son las pruebas de software, cuáles son sus objetivos, qué niveles y tipos existen, y por qué su planificación permite tomar decisiones técnicas más organizadas durante el aseguramiento de la calidad.

El estudio de este componente es importante porque permite al aprendiz reconocer que probar una solución tecnológica no consiste en realizar acciones al azar, sino en definir una estrategia estructurada que oriente el proceso de verificación y validación. A través de estos contenidos, el aprendiz podrá identificar la relación entre los requisitos del cliente, los estándares técnicos, los escenarios de prueba y los casos de prueba, comprendiendo cómo estos elementos aportan a la prevención de errores, la detección de defectos y la mejora de la experiencia del usuario final.

El componente se desarrollará mediante explicaciones conceptuales, ejemplos aplicados y recursos educativos virtuales que faciliten la comprensión progresiva de los temas. Inicialmente, se abordarán los conceptos, objetivos, alcance, principios e importancia de las pruebas de software; luego, se revisarán los niveles y tipos de prueba en relación con el ciclo de vida del desarrollo; posteriormente, se estudiarán los marcos de referencia, estándares y procedimientos técnicos; finalmente, se orientará la

construcción del plan de pruebas, considerando alcance, enfoque, recursos, cronograma, ítems, escenarios, nomenclaturas y casos de prueba.

Video 1. Fundamentos del plan de pruebas de software



[Enlace de reproducción del video](#)

Transcripción del video. Fundamentos del plan de pruebas de software

Fundamentos del plan de pruebas de software. En el sector tecnológico colombiano, la calidad del software es un factor clave para entregar aplicaciones confiables, seguras y útiles en empresas, instituciones educativas y servicios digitales.

Las pruebas de software permiten verificar y validar que una solución responda a los requisitos definidos. reduzca riesgos y aporte confianza antes de su implementación. Para el aprendiz, comprender este proceso fortalece su desempeño en roles relacionados con desarrollo, soporte, análisis de calidad y gestión de proyectos tecnológicos.

Transcripción del video. Fundamentos del plan de pruebas de software

Saber organizar pruebas mejora la empleabilidad porque las organizaciones requieren talento capaz de detectar defectos, documentar hallazgos y apoyar decisiones que eviten pérdidas económicas, reprocesos y afectaciones al usuario.

En este componente se abordan los fundamentos de las pruebas, los objetivos y el alcance, los niveles asociados al ciclo de vida, los tipos funcionales, no funcionales, estáticos y dinámicos, los estándares técnicos y la documentación requerida. Estos contenidos se articulan con el resultado de aprendizaje orientado a determinar el plan de pruebas de software de acuerdo con estándares y procedimientos técnicos.

Al finalizar se espera que el aprendiz reconozca cómo estructurar un plan de pruebas con alcance, enfoque, recursos, cronograma, ítems, escenarios, nomenclaturas y casos de prueba. Este aprendizaje favorece la entrega de soluciones digitales confiables, seguras, usables y alineadas con las necesidades del proyecto, fortaleciendo la calidad del servicio tecnológico en el territorio.

1. Fundamentos de las pruebas de software

El estudio de las pruebas de software debe iniciar con una comprensión profunda de su naturaleza. En algunos contextos formativos, se tiene la concepción errónea de que probar software consiste únicamente en “hacer clic en botones” para identificar si el sistema falla. Esta visión reduccionista impide dimensionar el verdadero valor de la disciplina.

En esta sección se aborda la fundamentación teórica necesaria para comprender el testing como una actividad analítica, estructurada y preventiva. Esta comprensión permite reconocer que las pruebas de software no son una acción improvisada, sino un proceso técnico que aporta información sobre la calidad, funcionalidad y confiabilidad de una solución tecnológica.

- **Valor de las pruebas de software**

- **Actividad analítica**

Las pruebas requieren analizar requisitos, condiciones, riesgos y resultados esperados antes de ejecutar cualquier validación.

- **Actividad estructurada**

El proceso de prueba se organiza mediante planes, escenarios, casos de prueba, criterios y evidencias.

- **Actividad preventiva**

Las pruebas ayudan a identificar defectos de manera temprana y a reducir riesgos antes de la entrega del producto.

- **Actividad de calidad**

El testing aporta información objetiva para valorar la confiabilidad, funcionalidad y utilidad del software.

1.1. Concepto, objetivos y alcance de las pruebas de software

Las pruebas de software se definen como un proceso técnico e investigativo llevado a cabo para proporcionar a los interesados, como clientes, desarrolladores y gerentes, información objetiva e independiente sobre la calidad del producto o servicio de software que se está evaluando.

No se trata de un evento aislado que ocurre al final del desarrollo, sino de un conjunto de actividades concurrentes que buscan verificar y validar que el sistema informático cumple con los requisitos técnicos, de negocio y normativos.

Desde una perspectiva pedagógica, el concepto de pruebas de software puede comprenderse mediante una analogía con la construcción de un edificio. Un ingeniero civil no espera a que el edificio esté terminado para comprobar si los cimientos soportan el peso; realiza cálculos, inspecciona los materiales y aplica pruebas de tensión durante todo el proceso.

De manera similar, en el desarrollo de software, las pruebas son evaluaciones continuas de los “materiales”, representados en el código fuente, y de los “planos”, representados en los requisitos del cliente, para asegurar que la aplicación construida sea sólida, segura y funcional. Comprender esta esencia es el primer paso para determinar un plan de pruebas coherente y preventivo.

Los objetivos principales de las pruebas de software incluyen los siguientes:

- **Encontrar defectos o bugs:** identificar errores, inconsistencias y vulnerabilidades antes de que el software sea entregado al usuario final.
- **Ganar confianza:** demostrar que el software cumple con sus especificaciones y que es seguro utilizarlo en un entorno de producción.
- **Proporcionar información para la toma de decisiones:** entregar reportes y métricas que permitan a los líderes del proyecto decidir si un producto está listo para salir al mercado o si requiere ajustes.
- **Prevenir defectos:** en enfoques ágiles y modernos, involucrar a los analistas de pruebas desde el diseño de los requisitos ayuda a evitar que los errores se introduzcan en el código.

El alcance de las pruebas define qué partes del sistema serán evaluadas y cuáles no, considerando las limitaciones de tiempo y presupuesto. Por ejemplo, en el desarrollo de un sitio web de comercio electrónico formativo, el alcance podría incluir la validación del carrito de compras y la pasarela de pagos, excluyendo temporalmente la sección de foros de la comunidad.

Delimitar el alcance es una habilidad analítica fundamental al momento de construir el plan de pruebas, porque permite establecer prioridades, organizar recursos y definir con claridad los límites del proceso de evaluación.

- **Conceptos clave de las pruebas de software**

- **Proceso técnico**

Las pruebas de software permiten evaluar un producto tecnológico mediante criterios, procedimientos y evidencias.

- **Información objetiva**

El proceso de prueba entrega datos útiles para clientes, desarrolladores y responsables del proyecto.

- **Verificación**

Permite comprobar si el sistema fue construido según los requisitos técnicos, de negocio y normativos establecidos.

- **Validación**

Permite confirmar si el sistema responde a las necesidades del cliente o del usuario final.

- **Alcance**

Define qué partes del sistema serán evaluadas y cuáles quedarán fuera del proceso de prueba.

1.2. Principios, importancia e impacto del software testing

Existen principios internacionalmente reconocidos, promovidos por entidades como el International Software Testing Qualifications Board (ISTQB), que todo profesional en formación debe interiorizar. Estos principios actúan como guías heurísticas que orientan el diseño y la ejecución de las pruebas, permitiendo comprender que el software testing no se limita a encontrar errores, sino que exige análisis, priorización, actualización permanente y comprensión del contexto en el que opera cada sistema.

- **Principios universales del software testing**

- **Las pruebas demuestran la presencia de defectos, no su ausencia**

Probar un sistema reduce la probabilidad de que existan fallos ocultos, pero incluso si no se encuentra ningún error, esto no es prueba absoluta de que el software sea perfecto.

- **Las pruebas exhaustivas son imposibles**

Probar todas las combinaciones posibles de datos de entrada y precondiciones es inviable en la mayoría de los sistemas. Por ello, se debe priorizar mediante el análisis de riesgos para optimizar los recursos.

- **Pruebas tempranas o Shift-Left Testing**

Las actividades de prueba deben comenzar lo antes posible en el ciclo de vida del software, idealmente desde la revisión de los documentos de requisitos. Corregir un error en la fase de diseño es mucho más económico que hacerlo cuando el sistema ya está programado.

- **Agrupamiento de defectos**

La regla del 80 / 20, conocida como principio de Pareto, suele aplicar al software: aproximadamente el 80 % de los defectos operativos se encuentran en el 20 % de los módulos críticos del sistema. Identificar estos módulos es clave para un plan de pruebas efectivo.

- **La paradoja del pesticida**

Si se repiten exactamente las mismas pruebas una y otra vez, eventualmente esos casos de prueba dejarán de encontrar nuevos defectos. Al igual que las plagas se

vuelven resistentes a un pesticida, el software puede dejar de revelar errores frente a pruebas repetitivas. Es necesario revisar y actualizar los escenarios de prueba periódicamente.

- **Las pruebas dependen del contexto**

No se prueba de la misma manera una aplicación móvil de entretenimiento que un software de control de tráfico aéreo. El rigor, las técnicas y los estándares varían según el tipo de sistema, el nivel de riesgo y el contexto de uso.

- **La falacia de la ausencia de errores**

Si el sistema construido es inestable, difícil de usar o no cumple con las necesidades de negocio del cliente, no importará que se hayan solucionado todos los defectos técnicos. Un software sin errores funcionales pero inútil para el usuario, sigue siendo un producto fallido.

A partir de estos principios, se comprende que las pruebas de software no solo permiten detectar errores técnicos, sino que también aportan valor al desarrollo tecnológico, la seguridad de la información y la experiencia del usuario. Su impacto se refleja en la reducción de riesgos, la toma de decisiones fundamentada y la entrega de soluciones más confiables. Por esta razón, el siguiente recurso presenta algunos aportes clave de las pruebas dentro de un proyecto de software.

- **Impacto de las pruebas del desarrollo tecnológico**

- a. Red de seguridad**

Las pruebas ayudan a detectar fallos antes de que afecten la operación, los datos o la experiencia de usuario final.

b. Responsabilidad técnica

El plan de pruebas permite tomar decisiones organizadas sobre qué evaluar, cómo hacerlo y con qué criterios.

c. Integridad de los datos

Una prueba bien planificada ayuda a reducir riesgos asociados con pérdida, alteración o exposición de información.

d. Calidad del producto

Los resultados de las pruebas aportan evidencias para mejorar la funcionalidad, confiabilidad y utilidad del software.

2. Niveles de pruebas de software y ciclo de vida

A partir de los fundamentos teóricos, es necesario comprender cómo se estructuran temporal y arquitectónicamente las pruebas. Los niveles de prueba organizan las evaluaciones en fases que corresponden a la evolución del software, desde sus unidades de código más pequeñas hasta el sistema completo en su entorno real.

Esta organización permite reconocer en qué momento del desarrollo se aplican determinadas pruebas y qué propósito cumple cada una dentro del aseguramiento de la calidad. De esta manera, los niveles de prueba ayudan a planificar el cronograma, los recursos, los escenarios y los casos de prueba de forma coherente con el avance del producto tecnológico.

2.1. Ciclo de vida del desarrollo de software y Modelo en V

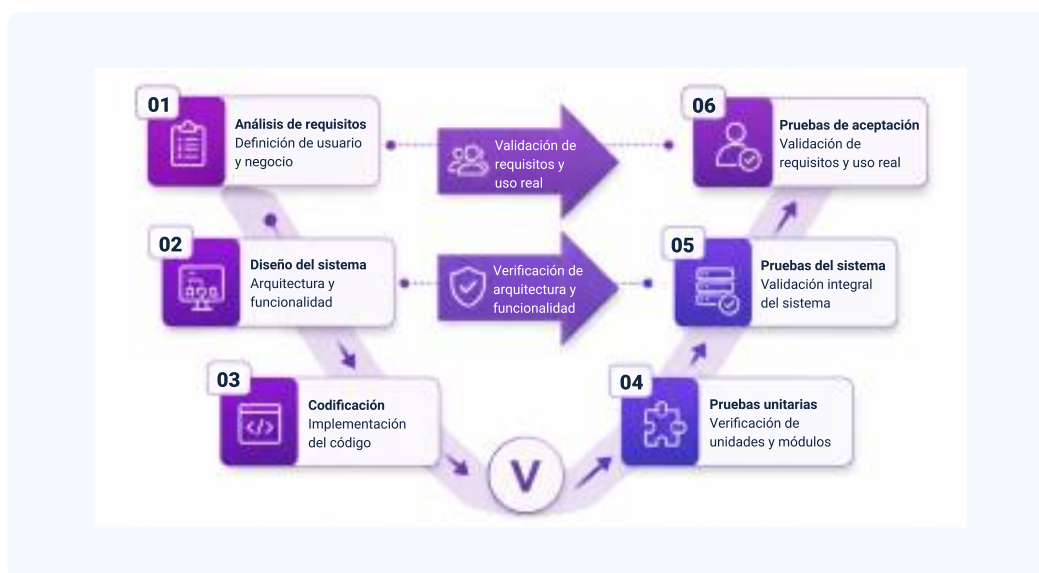
El ciclo de vida del desarrollo de software (SDLC, por sus siglas en inglés) es el marco metodológico que describe las etapas por las que pasa un producto informático desde su concepción hasta su retiro. Modelos tradicionales como el Modelo en Cascada, o su variante orientada a pruebas, el Modelo en V, establecen una relación simétrica entre las fases de desarrollo y los niveles de prueba.

Por otro lado, en marcos de trabajo ágiles, como Scrum, estos niveles no se presentan como fases aisladas, sino que se integran en iteraciones cortas o sprints. Independientemente de la metodología utilizada, se requiere interpretar el ciclo de vida para determinar cuándo aplicar cada nivel de prueba.

Esta comprensión es esencial para planificar el cronograma dentro del plan de pruebas, ya que permite relacionar los momentos del desarrollo con las actividades de verificación y validación correspondientes.

- **Relación entre ciclo de vida y niveles de prueba**
 - **Análisis de requisitos:** en esta etapa se identifican las necesidades del cliente, las condiciones del negocio y los requisitos que servirán como base para validar el producto.
 - **Diseño del sistema:** se define la arquitectura y funcionalidad del sistema. Esta información orienta la verificación de componentes, interfaces y comportamiento esperado.
 - **Codificación:** se implementa el código fuente. En esta etapa se relacionan las pruebas unitarias, orientadas a verificar unidades, métodos o funciones específicas.
 - **Integración y sistema:** los componentes se conectan y se evalúa el comportamiento del sistema completo. Aquí se relacionan las pruebas de integración y las pruebas del sistema.
 - **Aceptación y uso real:** el producto se valida frente a los requisitos y necesidades del usuario final o del cliente antes de su liberación o implementación.

Figura 1. Diagrama del Modelo en V del ciclo de vida del software



2.2. Pruebas unitarias, de integración, del sistema y de aceptación

Los niveles de prueba permiten evaluar el software de manera progresiva. Cada nivel cumple una función específica dentro del ciclo de vida: primero se verifica la lógica interna de unidades pequeñas, luego la comunicación entre componentes, después el sistema completo y, finalmente, la aceptación por parte del cliente o usuario final.

- **Niveles de prueba del software**

- **Pruebas unitarias**

Las pruebas unitarias constituyen el nivel más bajo y granular de la evaluación de software. Su objetivo es aislar y verificar la funcionalidad de un fragmento de código específico, denominado “unidad”, que generalmente corresponde a una función, un método o una clase en la programación orientada a objetos.

En la práctica formativa y profesional, estas pruebas suelen ser escritas y ejecutadas por los desarrolladores utilizando marcos de trabajo técnicos, como JUnit

para Java o PyTest para Python. Su propósito no es verificar el sistema completo, sino garantizar que la lógica interna de cada componente funcione como se diseñó.

Por ejemplo, en el desarrollo de un aula web académica, una prueba unitaria evaluaría exclusivamente si la función matemática que calcula el promedio de notas de un estudiante devuelve el resultado correcto con ciertos parámetros de entrada, sin importar si la interfaz gráfica está conectada.

- **Pruebas de integración**

Incluso si todas las unidades de código funcionan correctamente de manera aislada, los problemas suelen surgir cuando estas unidades se conectan para interactuar entre sí. Las pruebas de integración se enfocan en evaluar la interfaz y el flujo de datos entre módulos o componentes de software.

Estas pruebas buscan detectar fallos en la comunicación, como formatos de datos incompatibles o llamadas a funciones incorrectas. Existen diferentes enfoques para la integración, como Big Bang, ascendente (bottom-up) o descendente (top-down).

En el escenario de una plataforma académica, una prueba de integración evaluaría si el módulo de “cálculo de promedios” se comunica correctamente con el módulo de “base de datos de estudiantes” para extraer y actualizar la información sin generar acoplamientos de conexión.

- **Pruebas del sistema**

Cuando todos los componentes se han integrado, se procede a las pruebas del sistema. Este nivel evalúa el software en su totalidad, como un producto completo y

ensamblado, para verificar que cumple con los requerimientos funcionales y no funcionales definidos por el cliente.

Las pruebas del sistema se realizan en un entorno de pruebas que debe simular, lo más fielmente posible, el entorno de producción real. En este nivel se asume la perspectiva del usuario final y se evalúa el comportamiento del software en escenarios completos.

Continuando con el ejemplo de la plataforma académica, la prueba del sistema implicaría iniciar sesión como docente, ingresar notas en múltiples asignaturas, generar un reporte en PDF y verificar que el sistema no se congela durante el proceso. Este nivel debe contemplarse detalladamente al elaborar los escenarios y nomenclaturas del plan de prueba.

- **Pruebas de implementación y aceptación del usuario**

Las pruebas de aceptación son el último nivel antes de que el software sea liberado o implementado oficialmente. Su propósito principal no es buscar nuevos defectos técnicos, que debieron resolverse en niveles anteriores, sino validar que el sistema satisface las necesidades de negocio y genera confianza para su despliegue.

Generalmente, estas pruebas, conocidas como UAT o User Acceptance Testing, son realizadas por el cliente, los usuarios finales o representantes del negocio, con el acompañamiento del equipo técnico. Su división con frecuencia en pruebas alfa, realizadas en las instalaciones de los desarrolladores, y pruebas beta, realizadas por usuarios en sus propios entornos.

La determinación de este nivel dentro del procedimiento técnico de prueba es crucial, ya que representa la firma de conformidad que certifica que el software entregado aporta el valor esperado.

3. Tipos de pruebas de software

Mientras que los niveles de prueba indican en qué momento del ciclo de vida se realiza la evaluación, los tipos de prueba definen el enfoque o el objetivo específico de lo que se está evaluando. Diferenciar los tipos de pruebas de software permite seleccionar las herramientas, técnicas y metodologías adecuadas para estructurar un plan de pruebas robusto y pertinente a las necesidades del proyecto.

Esta clasificación facilita la toma de decisiones dentro del proceso de aseguramiento de la calidad, ya que no todos los sistemas requieren las mismas pruebas ni con el mismo nivel de profundidad. Por ello, es necesario reconocer qué se evalúa, cómo se evalúa y qué información aporta cada tipo de prueba al análisis del producto tecnológico.

3.1. Pruebas funcionales y no funcionales

Las pruebas funcionales se centran en evaluar “qué hace” el sistema. Su objetivo principal es verificar que las funciones del software operen en concordancia con los requisitos especificados por el cliente o el analista de negocio. En este tipo de pruebas, el código interno generalmente se ignora, debido a que se trabaja desde un enfoque de caja negra, centrado en suministrar datos de entrada y validar que los datos de salida o el comportamiento del sistema sean los esperados.

Para ilustrar este concepto en un contexto de formación profesional, puede tomarse como referencia el diseño de un Sistema de Gestión de Aprendizaje (LMS, por sus siglas en inglés). Una prueba funcional evaluaría si un aprendiz, al ingresar su número de documento y contraseña correctos, logra acceder a su panel de cursos.

Asimismo, validaría que el sistema deniegue el acceso y muestre un mensaje de error claro si la contraseña es incorrecta.

En el plan de pruebas, las pruebas funcionales constituyen el núcleo de la matriz de casos de prueba, pues garantizan que la aplicación cumple con su propósito operativo fundamental.

A diferencia de las pruebas funcionales, las pruebas no funcionales se enfocan en “cómo lo hace” el sistema. Un software puede cumplir con todas sus funciones, pero si es extremadamente lento, vulnerable a ataques informáticos o difícil de comprender para el usuario, será considerado un producto defectuoso.

Las pruebas no funcionales evalúan atributos de calidad del sistema, como rendimiento, carga, seguridad, usabilidad y accesibilidad. Incluir estos requerimientos en el plan de pruebas es fundamental, porque omitirlos puede llevar a sistemas funcionales en teoría, pero poco eficientes, inseguros o difíciles de usar en condiciones reales.

- **Pruebas funcionales y no funcionales**

- **Pruebas funcionales:** evalúan “qué hace” el sistema. Permiten verificar si las funciones del software cumplen con los requisitos definidos por el cliente o el analista de negocio.
- **Enfoque de caja negra:** en las pruebas funcionales, generalmente no se analiza el código interno. Se ingresan datos y se valida si la respuesta del sistema coincide con el resultado esperado.

- **Ejemplo funcional:** en un LMS, una prueba funcional puede validar si un aprendiz accede correctamente a su panel de cursos al ingresar credenciales válidas, o si recibe un mensaje de error cuando la contraseña es incorrecta.
- **Pruebas no funcionales:** evalúan “cómo lo hace” el sistema. Permiten revisar atributos de calidad como rendimiento, seguridad, usabilidad, accesibilidad y comportamiento bajo condiciones de alta demanda.
- **Importancia en el plan:** las pruebas funcionales y no funcionales deben considerarse en el plan de pruebas para evaluar tanto el cumplimiento de requisitos como la calidad de la experiencia de uso.

Dentro de las pruebas no funcionales, es necesario precisar qué atributos de calidad serán evaluados, ya que no basta con que el software cumpla sus funciones principales. También debe responder adecuadamente frente a condiciones de uso reales, proteger la información, facilitar la interacción de las personas usuarias y permitir el acceso en condiciones de equidad. Por ello, el siguiente recurso presenta algunos atributos que pueden incluirse en el plan de pruebas.

- **Atributos evaluados en las pruebas no funcionales**

- A. Rendimiento y carga**

Evalúa cómo se comporta el sistema bajo condiciones de alta demanda. Por ejemplo, en un portal educativo con inscripciones nacionales, una prueba de carga puede simular múltiples usuarios accediendo al mismo tiempo para comprobar que los servidores respondan adecuadamente.

B. Seguridad

Verifica que los datos confidenciales, como notas, información personal o certificados, estén protegidos contra accesos no autorizados.

C. Usabilidad

Analiza la facilidad con la que las personas interactúan con la interfaz del software, incluyendo usuarios con discapacidad visual, auditiva, cognitiva o motora.

D. Accesibilidad

Permite revisar si el sistema puede ser utilizado por la mayor cantidad posible de personas, considerando lectura con tecnologías de apoyo, navegación por teclado, contraste suficiente, textos alternativos y claridad en los mensajes.

3.2. Pruebas de revisión, estáticas y dinámicas

La industria del desarrollo de software clasifica las pruebas en dos grandes paradigmas metodológicos: pruebas estáticas y pruebas dinámicas. Esta clasificación permite diferenciar si la evaluación requiere ejecutar el código fuente o si puede realizarse mediante el análisis de documentos, modelos, diseños o código sin poner en funcionamiento el sistema.

Las pruebas dinámicas requieren la ejecución del código fuente; es decir, el software debe estar compilado y en funcionamiento para que el probador interactúe con él. Las pruebas funcionales y no funcionales mencionadas anteriormente son, en su mayoría, de naturaleza dinámica, porque permiten revisar el comportamiento del sistema durante su operación.

Por otro lado, las pruebas estáticas se realizan sin ejecutar el código. Su objetivo es examinar la documentación, los modelos de diseño o el código fuente para encontrar anomalías de forma temprana. Una de las técnicas más valiosas dentro de las pruebas estáticas son las revisiones.

Las revisiones consisten en reuniones estructuradas donde el equipo de desarrollo, analistas y clientes examinan documentos como el documento de requisitos o el diseño de la base de datos. Desde una perspectiva pedagógica, las revisiones funcionan como un ejercicio de aprendizaje colaborativo, porque permiten detectar ambigüedades, contradicciones o vacíos antes de avanzar hacia la codificación.

Por ejemplo, al revisar en conjunto el prototipo de un sitio web, el equipo puede identificar que un requisito es ambiguo o contradictorio antes de que un programador invierta tiempo escribiendo el código. Las pruebas estáticas representan una aplicación directa del principio de pruebas tempranas y contribuyen al ahorro de recursos financieros, técnicos y humanos.

- **Pruebas estáticas, dinámicas y de revisión**

- **Pruebas dinámicas**

Requieren ejecutar el código fuente. El software debe estar compilado y en funcionamiento para evaluar su comportamiento.

- **Pruebas estáticas**

No requieren ejecutar el código. Se centran en revisar documentación, modelos, diseños o código fuente para identificar anomalías tempranas.

- **Revisiones técnicas**

Son espacios estructurados donde el equipo examina documentos del proyecto, como requisitos, prototipos o diseños de base de datos.

- **Valor preventivo**

Permiten detectar errores antes de la programación o ejecución del sistema, reduciendo reprocesos y costos.

Las pruebas dinámicas evalúan el comportamiento del sistema en funcionamiento. Las pruebas estáticas permiten revisar documentos, diseños o código sin ejecutar el software. Ambas aportan información valiosa para construir un plan de pruebas más completo y preventivo.

4. Marcos de referencia y estándares en pruebas de software

El aseguramiento de la calidad del software no es una labor empírica basada en la intuición; es una disciplina de la ingeniería fundamentada en metodologías rigurosas. Para determinar el plan de pruebas, es necesario interpretar y aplicar principios, marcos de referencia y estándares internacionales.

Estos lineamientos proporcionan un lenguaje común y estructurado que profesionaliza la labor del analista de pruebas, facilita la documentación del proceso y permite que los resultados sean comprensibles para los diferentes integrantes del equipo de desarrollo.

4.1. Estándares, técnicas y procedimientos técnicos

Los estándares son conjuntos de reglas, directrices y definiciones avaladas por organizaciones internacionales para unificar la manera en que se desarrollan y prueban los sistemas. Históricamente, el estándar IEEE 829 fue uno de los referentes principales para la documentación de pruebas de software. Sin embargo, en la actualidad, la norma ISO/IEC/IEEE 29119 ha tomado protagonismo, al ofrecer un marco que abarca conceptos, procesos, documentación y técnicas de prueba aplicables tanto a metodologías tradicionales como ágiles.

Paralelamente, instituciones como el ISTQB, sigla de International Software Testing Qualifications Board, no emiten normativas legales, pero proporcionan el marco de conocimientos, conocido como Syllabus, más adoptado a nivel global. En el ámbito formativo, alinear el conocimiento con estos estándares permite que el vocabulario técnico y los formatos utilizados sean comprendidos por cualquier equipo

de desarrollo. La adherencia a estándares no busca burocratizar el proceso, sino asegurar trazabilidad, repetibilidad y rigor técnico.

- **Referentes para estandarizar las pruebas**

- **Estándares:** son reglas, directrices y definiciones que permiten unificar la forma en que se desarrollan, documentan y ejecutan las pruebas de software.
- **IEEE829:** fue uno de los referentes históricos para la documentación de pruebas de software, especialmente en la organización de artefactos y formatos asociados al proceso.
- **ISO/IEC/IEEE 29119:** proporciona un marco amplio para conceptos, procesos, documentación y técnicas de prueba, aplicable en metodologías tradicionales y ágiles.
- **ISTQB:** el International Software Testing Qualifications Board aporta un marco de conocimientos ampliamente reconocido para orientar el vocabulario y las buenas prácticas del testing.
- **Rigor técnico:** el uso de estándares fortalece la trazabilidad, la repetibilidad y la comprensión del proceso de pruebas por parte del equipo de desarrollo.

Un procedimiento técnico en pruebas de software es la secuencia lógica y ordenada de pasos necesarios para preparar, ejecutar y evaluar las pruebas. La determinación de este procedimiento requiere seleccionar técnicas estandarizadas acordes con el contexto del proyecto.

Por ejemplo, al definir técnicas de diseño de casos de prueba, los estándares y buenas prácticas orientan el uso de métodos formales como la “partición de clases de equivalencia” o el “análisis de valores límite”. En lugar de ingresar datos al azar en un

formulario web, se aplica un procedimiento técnico para identificar qué subconjuntos de datos son representativos y probarlos de manera organizada, optimizando el tiempo y maximizando la cobertura.

Comprender el procedimiento técnico permite pasar de pruebas empíricas a experimentos formales de validación de software, con criterios definidos, datos seleccionados y resultados verificables.

- **Procedimiento técnico para diseñar pruebas**

- A. **Preparar:** se identifican requisitos, condiciones del proyecto, datos de prueba, entorno y criterios que orientan la evaluación.
- B. **Seleccionar técnicas:** se eligen técnicas acordes con el contexto, como partición de clases de equivalencia o análisis de valores límite.
- C. **Ejecutar:** se aplican los casos de prueba definidos, utilizando datos representativos y condiciones previamente establecidas.
- D. **Evaluar:** se comparan los resultados obtenidos con los resultados esperados para determinar si el software cumple con lo definido.

4.2. Documentación, gestión de configuración y lecciones aprendidas

La documentación técnica es la evidencia tangible del proceso de testing. Sin documentación, es imposible demostrar qué se probó, cómo se probó y qué resultados se obtuvieron, lo cual invalida el esfuerzo realizado ante auditorías de calidad. Los estándares exigen la creación de artefactos como la política de pruebas, la estrategia, el plan maestro, los casos de prueba y los informes de incidentes.

Estrechamente ligada a la documentación se encuentra la gestión de la configuración. Este concepto hace referencia al control de las versiones tanto del software que se está probando como de los documentos asociados. En un entorno real, el código cambia constantemente. Si se reporta un defecto, debe indicarse exactamente en qué versión del software, por ejemplo, versión 1.2.4, y en qué entorno, como desarrollo o producción, ocurrió el fallo.

Sin una gestión de la configuración estandarizada, el equipo de desarrollo puede perder tiempo intentando reproducir errores en versiones incorrectas del sistema. Por esta razón, documentar y controlar versiones no es una tarea secundaria, sino una condición necesaria para asegurar trazabilidad y confiabilidad en el proceso de pruebas.

- **Documentación y gestión de configuración**

- **Documentación técnica**

Permite demostrar qué se probó, cómo se probó y qué resultados se obtuvieron durante el proceso de testing.

- **Artefactos de prueba**

Incluyen política de pruebas, estrategia, plan maestro, casos de prueba e informes de incidentes.

- **Control de versiones**

Permite identificar la versión exacta del software y de los documentos asociados al momento de reportar un defecto.

- **Trazabilidad del fallo**

Facilita reproducir errores en el entorno y versión correspondientes, evitando confusiones durante la corrección.

La mejora continua es el pilar de cualquier marco de calidad. Una vez finaliza un ciclo de pruebas, el estándar indica que el equipo debe reflexionar sobre el proceso ejecutado. El manejo de las lecciones aprendidas consiste en documentar los aspectos positivos, como técnicas que funcionaron o herramientas que agilizaron el trabajo, y los aspectos por mejorar, como cuellos de botella, errores no detectados o fallas de comunicación.

Desde un enfoque educativo, esta etapa fomenta la metacognición en el aprendiz. Al analizar retrospectivamente un proyecto simulado, se identifican oportunidades de optimización y se proponen planes de mejora concretos, desarrollando una postura analítica y crítica frente al propio desempeño. Estas lecciones aprendidas se convierten en activos organizacionales valiosos para afrontar futuros proyectos con mayor grado de madurez técnica.

- **Ruta para gestionar lecciones aprendidas**

- a. Revisar el proceso ejecutado**

Se analiza cómo se prepararon, ejecutaron y documentaron las pruebas durante el ciclo de trabajo.

- b. Identificar aciertos**

Se registran técnicas, herramientas, decisiones o prácticas que facilitaron el proceso y generaron buenos resultados.

c. Reconocer aspectos por mejorar

Se documentan cuellos de botella, errores no detectados, fallas de comunicación o dificultades en la ejecución.

d. Proponer mejoras

Se formulan acciones concretas para optimizar futuros ciclos de prueba y fortalecer la madurez técnica del equipo.

5. Plan de pruebas de software (test plan)

La consolidación de los saberes conceptuales y procedimentales abordados hasta este punto se materializa en uno de los instrumentos más importantes del analista de QA: el plan de pruebas. Construir este documento es el objetivo central del primer resultado de aprendizaje de este componente formativo.

No se trata simplemente de diligenciar una plantilla, sino de tomar decisiones estratégicas fundamentadas en los requerimientos del cliente, los recursos disponibles y los riesgos inherentes al desarrollo. Por esta razón, el plan de pruebas funciona como una guía técnica que orienta qué se va a probar, cómo se realizará el proceso, con qué recursos y bajo qué criterios se evaluarán los resultados.

El plan de pruebas no es solo un formato administrativo. Es una estrategia técnica que permite organizar el proceso de evaluación del software, definir prioridades, anticipar riesgos y asegurar que las pruebas respondan a los requisitos del proyecto.

5.1. Concepto, propósito, alcance y enfoque del plan de pruebas

El plan de pruebas, conocido en inglés como test plan, es el documento maestro que describe el alcance, el enfoque, los recursos y el cronograma de las actividades de prueba previstas. Su propósito es identificar explícitamente los elementos de software que serán probados, las características que se evaluarán, las responsabilidades del equipo y los riesgos asociados a la ejecución de las pruebas.

En términos prácticos, el plan de pruebas es la hoja de ruta que permite que el equipo técnico y gerencial comparta una misma visión sobre cómo se asegurará la calidad del producto. La industria actual, basada en el estándar ISO/IEC/IEEE 29119-3,

propone formatos estructurados que evitan la ambigüedad y reducen el riesgo de omitir fases críticas.

Aunque en metodologías ágiles el plan de pruebas suele ser más conciso y dinámico que en metodologías tradicionales, su esencia estratégica y analítica permanece. Por ello, es necesario reconocer su estructura formal para adaptarla con criterio al contexto de cada proyecto tecnológico.

Antes de continuar con la definición del alcance y el enfoque del plan de pruebas, acceda al pódcast Plan de pruebas: la ruta para entregar software confiable. Este recurso permite comprender, mediante una conversación sencilla, por qué el plan de pruebas no debe asumirse como un formato, sino como una estrategia técnica para organizar la evaluación del software, anticipar riesgos y tomar decisiones durante el aseguramiento de la calidad.

Transcripción del pódcast: Plan de pruebas: la ruta para entregar software confiable

Hombre: ¡Bienvenidos a este espacio formativo! En este episodio hablaremos sobre la importancia del plan de pruebas, sus principales componentes y su aporte a la gestión de proyectos de software. ¡Bienvenidos!

Mujer: imagine que un equipo construye una aplicación para gestionar notas académicas y decide probarla solo al final, cuando ya casi debe entregarla. ¿Qué podría pasar si no existe una ruta clara para saber qué se prueba, cuándo se prueba y con qué criterios?

Hombre: pues que podrían aparecer errores difíciles de corregir, retrasos en la entrega y dudas sobre la calidad del producto. Por eso existe el plan de pruebas, también conocido como test plan.

Mujer: este documento organiza el proceso de evaluación del software, define el alcance, el enfoque, los recursos y el cronograma, y permite que el equipo técnico tenga una misma visión sobre cómo asegurar la calidad.

Hombre: entonces, ¿el plan de pruebas no es solo una plantilla para diligenciar?

Mujer: exacto. El plan de pruebas es una hoja de ruta técnica. Primero, ayuda a definir el alcance, es decir, qué funcionalidades serán evaluadas y cuáles quedarán por fuera en una fase determinada. Segundo, permite establecer el enfoque: si se aplicarán pruebas manuales, automatizadas, de caja negra, de caja blanca o una combinación de ellas. Tercero, facilita organizar recursos, tiempos, responsabilidades, escenarios y casos de prueba para evitar improvisaciones.

Hombre: ¿y cómo se relaciona esto con el trabajo real en un proyecto tecnológico?

Mujer: pensemos en un portal de noticias. Si el equipo define que en la primera fase se probarán la publicación y visualización de artículos, pero no el módulo de suscripciones de pago, el alcance queda claro. Esto evita confusiones. También permite estimar recursos, preparar datos de prueba y organizar el cronograma. Si el plan está bien elaborado, el equipo puede detectar defectos, documentar resultados y tomar decisiones con mayor seguridad.

Hombre: ¿Qué puede salir bien cuando se aplica correctamente el plan de pruebas? ¿Y qué puede salir mal si no se tiene en cuenta?

Mujer: Cuando se aplica correctamente, el equipo sabe qué evaluar, cómo hacerlo y qué resultado esperar. Además, se mejora la trazabilidad entre requisitos, escenarios y casos de prueba. En cambio, si no se define un plan, pueden quedar funcionalidades críticas sin revisar, pueden duplicarse esfuerzos o pueden reportarse defectos sin información suficiente para reproducirlos. Esto afecta la calidad del software y la confianza del cliente o usuario final.

Hombre: en pocas palabras, un buen plan de pruebas convierte la evaluación del software en un proceso ordenado, verificable y útil para tomar decisiones.

Mujer: así es. Por eso, al estudiar este tema, es clave revisar cómo se definen el alcance, el enfoque, los recursos, el cronograma, los ítems, los escenarios y los casos de prueba. Estos elementos permiten construir un plan de pruebas coherente con los requisitos del proyecto y con el propósito de entregar soluciones de software más confiables, seguras y alineadas con las necesidades del usuario.

Hombre: en conclusión, cuando las pruebas se planifican correctamente, los equipos pueden detectar defectos con mayor oportunidad, documentar resultados de manera estructurada y tomar decisiones con mayor seguridad.

Mujer: gracias por acompañarnos. Nos escuchamos en el próximo episodio.

El primer paso para construir el plan es delimitar el alcance. Esto implica listar con precisión las funcionalidades del sistema que serán objeto de pruebas, es decir, el alcance dentro de los límites, y justificar qué componentes no serán probados y por

qué, es decir, el alcance fuera de los límites. Por ejemplo, en la primera fase de desarrollo de un portal de noticias, el alcance puede abarcar la publicación y visualización de artículos, dejando por fuera, para una fase posterior, el módulo de suscripciones de pago.

Posteriormente, se define el enfoque o la estrategia. Esto responde al “cómo” se llevarán a cabo las pruebas. En este punto se determina si se dará prioridad a pruebas automatizadas o manuales, qué niveles de prueba se aplicarán, como unitarias, de integración o del sistema, y qué metodologías se utilizarán, como caja negra o caja blanca.

- **Elementos iniciales del plan de pruebas**

- **Conceptos:** El plan de pruebas o test plan es el documento maestro que organiza el alcance, el enfoque, los recursos y el cronograma de las actividades de prueba.
- **Propósito:** permite identificar qué elementos del software serán probados, qué características se evaluarán, quiénes serán responsables y qué riesgos pueden afectar la ejecución.
- **Formato estructurado:** los formatos basados en estándares, como ISO/IEC/IEEE 29119-3, ayudan a evitar ambigüedades y a no omitir fases críticas del proceso.
- **Alcance:** define qué funcionalidades serán evaluadas y qué componentes quedarán fuera del proceso, con su respectiva justificación.
- **Enfoque:** establece cómo se realizarán las pruebas: manuales o automatizadas, niveles aplicables y técnicas como caja negra o caja blanca.

5.2. Recursos, cronograma, ítems, escenarios y casos de prueba

La asignación de recursos exige una planeación realista. Se deben identificar los recursos humanos, como la cantidad de analistas requeridos y su nivel de experiencia; los recursos de hardware, como servidores o dispositivos móviles para pruebas de compatibilidad; y los recursos de software, como herramientas de gestión de defectos, entornos de automatización y bases de datos de prueba.

Por ejemplo, herramientas como Jira pueden apoyar la gestión de defectos y el seguimiento de incidencias durante el proceso de pruebas. Una planeación deficiente de recursos puede generar retrasos en las entregas de software, reprocesos y dificultades para ejecutar las pruebas previstas.

La elaboración del cronograma dentro del plan de pruebas trasciende la simple asignación de fechas en un calendario. Requiere sincronizar las actividades de prueba con los hitos del desarrollo de software. Para ello, se debe estimar el esfuerzo necesario para diseñar los casos de prueba, configurar el entorno, ejecutar las pruebas y documentar los resultados.

Un cronograma realista contempla márgenes para la corrección de defectos; es decir, asume que se encontrarán errores y que el equipo de desarrollo necesitará tiempo para solucionarlos antes de una segunda ronda de pruebas, conocidas como pruebas de regresión.

Junto con el tiempo, se deben definir los ítems de prueba. Un ítem es un componente específico del software, un módulo o un documento que será sometido a evaluación. Por ejemplo, en un sistema de información para una biblioteca virtual, un ítem de prueba podría ser el “Módulo de Búsqueda de Catálogo”.

A partir de los ítems, se estructuran los escenarios de prueba. Un escenario es una descripción de alto nivel que responde a la pregunta “¿qué se va a probar?” desde la perspectiva del flujo del usuario. Siguiendo el ejemplo de la biblioteca virtual, un escenario de prueba sería: “Verificar que un usuario registrado pueda buscar un libro por autor, agregarlo a sus favoritos y descargar el archivo PDF asociado”.

La correcta estructuración de escenarios garantiza la trazabilidad con los requisitos del cliente, asegurando que ninguna necesidad de negocio quede sin ser validada. Además, promueve una visión sistémica de la aplicación, al comprenderla como un conjunto de flujos de trabajo interconectados y no solo como pantallas aisladas.

- **Ruta para estructurar el plan de pruebas**

- a. Asignar recursos**

Identificar recursos humanos, de hardware y de software necesarios para preparar, ejecutar y documentar las pruebas.

- b. Elaborar el cronograma**

Sincronizar las actividades de prueba con los hitos del desarrollo, considerando diseño de casos, configuración del entorno, ejecución, documentación y corrección de defectos.

- c. Definir ítems de prueba**

Seleccionar módulos, componentes o documentos que serán sometidos a evaluación dentro del proceso de pruebas.

d. Estructurar escenarios

Describir de manera general qué se va a probar, tomando como referencia los flujos del usuario y los requisitos del cliente.

Para mantener el orden y la trazabilidad en proyectos tecnológicos que pueden involucrar cientos o miles de evaluaciones, el estándar exige el uso de nomenclaturas. Una nomenclatura es una convención de nombres o códigos de identificación únicos asignados a cada elemento del proceso de pruebas.

Por ejemplo, en lugar de llamar a una prueba “Prueba de inicio de sesión”, se le puede asignar un código estructurado como SIS-FUN-LOG-001, que corresponde a Sistema - Funcional - Login - Caso 01. Esta práctica fortalece el rigor administrativo necesario para gestionar configuraciones complejas y permite que cualquier integrante del equipo ubique rápidamente un caso específico en la matriz de diseño.

A partir de los escenarios y utilizando la nomenclatura establecida, se procede al diseño detallado de los casos de prueba. Mientras que el escenario describe el “qué”, el caso de prueba detalla el “cómo”. Un caso de prueba formalmente estructurado es el nivel más específico de la planeación y debe contener, como mínimo, los siguientes elementos procedimentales.

- **Elementos de un caso de prueba**

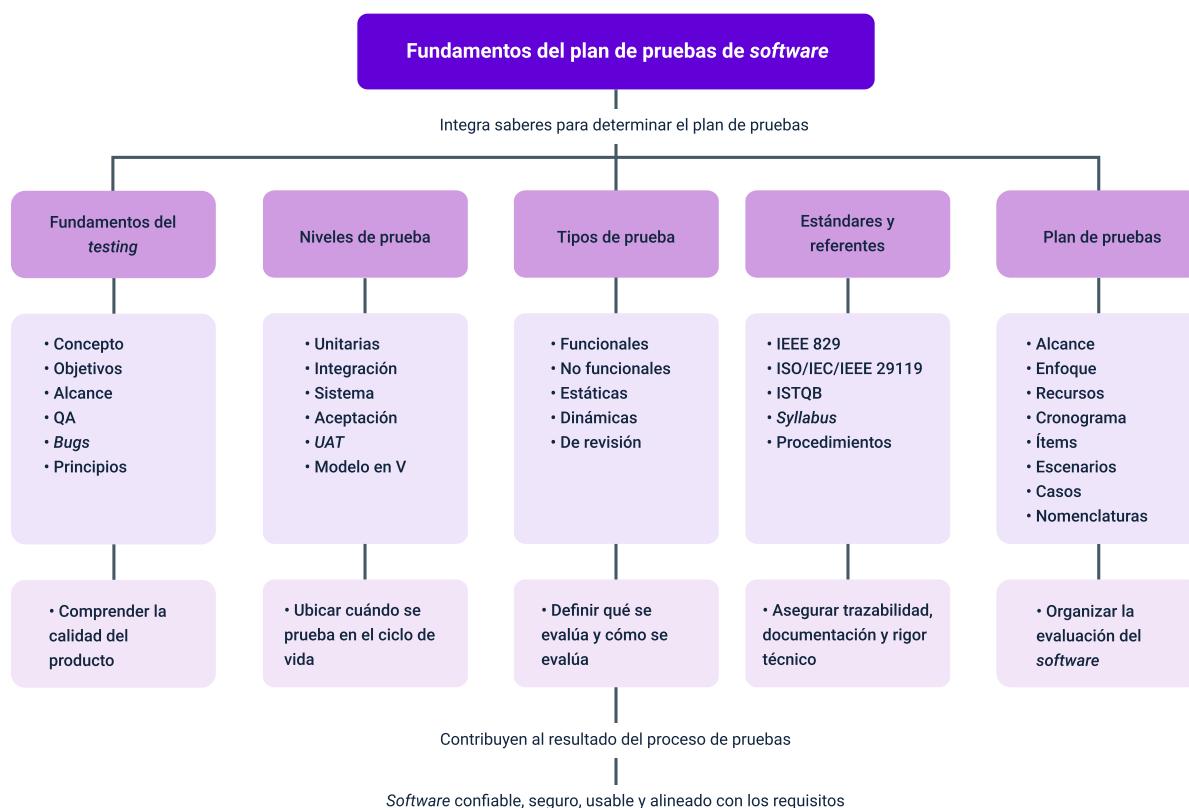
- **Identificador único:** corresponde a la nomenclatura asignada al caso de prueba. Permite ubicarlo, diferenciarlo y relacionarlo con un escenario o requisito específico.
- **Propósito o descripción:** indica el objetivo específico de la validación que se realizará sobre una funcionalidad, flujo o condición del sistema.

- **Precondiciones:** describe el estado en el que debe estar el sistema antes de iniciar la prueba. Por ejemplo: “El usuario debe estar previamente registrado en la base de datos”.
- **Pasos de ejecución:** presenta las instrucciones operativas que permiten ejecutar la prueba de forma ordenada y repetible.
- **Datos de prueba:** incluye los valores exactos que se ingresarán durante la prueba. Por ejemplo: “Usuario: admin@sena.edu.co, clave: 12345”.
- **Resultado esperado:** describe el comportamiento que el sistema debe presentar según los requisitos. Por ejemplo: “El sistema redirige al panel de control y muestra un mensaje de bienvenida”.

La capacidad de redactar casos de prueba claros, atómicos y repetibles evidencia la asimilación del resultado de aprendizaje. Un caso de prueba bien diseñado permite que cualquier persona, incluso ajena al proyecto, pueda ejecutar la prueba y llegar a la misma conclusión, eliminando la ambigüedad y profesionalizando el aseguramiento de la calidad del software.

Síntesis

Las pruebas de software constituyen un proceso técnico, analítico y preventivo que permite verificar y validar la calidad de una solución tecnológica. En este componente formativo se integran los fundamentos del testing, los niveles y tipos de pruebas, los estándares de referencia y los elementos necesarios para construir un plan de pruebas. Estos aspectos permiten organizar escenarios, casos, recursos, cronogramas y criterios de evaluación, con el fin de reducir riesgos, detectar defectos y aportar a la entrega de productos de software más confiables, seguros y alineados con las necesidades del usuario y del proyecto.



Glosario

Aseguramiento de la calidad (QA): conjunto de procesos sistemáticos destinados a garantizar que un producto o servicio de software cumpla con los estándares de calidad definidos durante su ciclo de vida.

Caso de prueba (test case): conjunto detallado de precondiciones, datos de entrada, acciones operativas y resultados esperados, diseñado para validar un objetivo o requerimiento específico del sistema.

Defecto (bug): imperfección o fallo en el código, diseño o requisitos de un software que causa que el sistema no funcione de acuerdo con sus especificaciones.

Entorno de pruebas: configuración específica de hardware, software, red y datos, estructurada para ejecutar las evaluaciones sin afectar el ambiente de producción de los usuarios finales.

Escenario de prueba: descripción de alto nivel de una funcionalidad que debe ser evaluada desde la perspectiva del flujo de uso del cliente final.

Gestión de la configuración: proceso técnico orientado a identificar, controlar y rastrear las versiones exactas de los componentes de software y la documentación a lo largo del tiempo.

ISTQB: sigla de International Software Testing Qualifications Board, organización global que define estándares, certificaciones y esquemas de conocimiento, como el Syllabus, para la profesión de pruebas de software.

Nomenclatura: sistema o conjunto de reglas establecidas para asignar códigos identificadores únicos a elementos como requisitos, escenarios, casos de prueba y reportes de incidentes.

Plan de pruebas (test plan): documento técnico y de gestión que establece la estrategia, el alcance, el cronograma y los recursos necesarios para asegurar la calidad de un proyecto tecnológico.

Pruebas de aceptación (UAT): evaluaciones formales llevadas a cabo por el cliente o los usuarios finales para determinar si el sistema satisface sus necesidades de negocio antes de la implementación real.

Pruebas de caja blanca: técnica de evaluación que se basa en el conocimiento de la estructura interna y la lógica del código fuente del software.

Pruebas de caja negra: técnica de prueba centrada en las entradas y salidas de la aplicación, sin considerar la estructura interna del código.

Pruebas del sistema: fase en la que se evalúa de manera integral el software completamente ensamblado para verificar que cumple con los requisitos funcionales y no funcionales.

Pruebas dinámicas: evaluaciones que requieren que el software sea compilado y ejecutado para analizar su comportamiento en tiempo real.

Pruebas estáticas: evaluaciones analíticas de la documentación, los requerimientos o el código fuente que se realizan sin ejecutar el programa.

Pruebas funcionales: evaluaciones diseñadas para verificar que el sistema realiza adecuadamente las funciones o tareas especificadas en los requisitos del cliente.

Pruebas unitarias: evaluaciones técnicas del nivel más granular, enfocadas en verificar el correcto funcionamiento aislado de funciones, métodos o pequeñas fracciones de código.

Referencias bibliográficas

Black, R. (2020). Agile Testing Foundations: An ISTQB Foundation Level Agile Tester Guide. BCS, The Chartered Institute for IT.

International Organization for Standardization [ISO]. (2021). Software and systems engineering -- Software testing -- Part 3: Test documentation (ISO/IEC/IEEE Standard 29119-3:2021).

International Software Testing Qualifications Board [ISTQB]. (2023). Certified Tester Foundation Level (CTFL) Syllabus v4.0. <https://www.istqb.org/>

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

Sommerville, I. (2021). Engineering Software Products: An Introduction to Modern Software Engineering. Pearson Education.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Edison Eduardo Mantilla Cuadros	Responsable de línea de producción	Centro Agroturístico - Regional Santander
Carlos Andres Bonza Reyes	Experto temático TIC	Centro Agroturístico - Regional Santander
Angelica Varon Quintero	Evaluadora instruccional	Centro Agroturístico - Regional Santander
Julian Fernando Vanegas Vega	Diseñador de contenidos	Centro Agroturístico - Regional Santander
Leonardo Castellanos Rodriguez	Desarrollador full stack	Centro Agroturístico - Regional Santander
Maria Alejandra Vera Briceño	Animadora y productora audiovisual	Centro Agroturístico - Regional Santander
Laura Paola Gelvez Manosalva	Validadora y vinculadora de recursos educativos digitales	Centro Agroturístico - Regional Santander
Sandra Liliana Cristancho Cruz	Evaluadora de contenidos inclusivos y accesibles	Centro Agroturístico - Regional Santander